
HEBBIAN LEARNING WITH GRADIENTS: HEBBIAN CONVOLUTIONAL NEURAL NETWORKS WITH MODERN DEEP LEARNING FRAMEWORKS

 **Thomas Miconi**

ML Collective

thomas.miconi@gmail.com

ABSTRACT

Deep learning networks generally use **non-biological learning methods**. By contrast, networks based on more biologically plausible learning, such as Hebbian learning, **show comparatively poor performance and difficulties of implementation**. Here we show that Hebbian learning in hierarchical, convolutional neural networks can be implemented almost trivially with modern deep learning frameworks, **by using specific losses whose gradients produce exactly the desired Hebbian updates**. We provide expressions whose gradients exactly implement a plain Hebbian rule ($\Delta w \propto xy$), Grossberg’s instar rule ($\Delta w \propto y(x - w)$), and Oja’s rule ($\Delta w \propto y(x - yw)$). As an application, we build Hebbian convolutional multi-layer networks for object recognition. We observe that higher layers of such networks tend to learn large, simple features (Gabor-like filters and blobs), explaining the previously reported decrease in decoding performance over successive layers. To combat this tendency, we introduce interventions (denser activations with sparse plasticity, pruning of connections between layers) which result in sparser learned features, massively increase performance, and allow information to increase over successive layers. **We hypothesize that more advanced techniques (dynamic stimuli, trace learning, feedback connections, etc.), together with the massive computational boost offered by modern deep learning frameworks, could greatly improve the performance and biological relevance of multi-layer Hebbian networks.**

1 Introduction

Recent advances in deep learning have greatly improved the state of the art in visual object recognition. Deep learning networks take inspiration from **features of the ventral visual cortex (including a hierarchical organization in successive layers and a topological organization of connections)**. Their internal activity shows similarities, as well as differences, with neural responses to natural images [Yamins and DiCarlo, 2016]. One important difference is that deep learning networks are usually trained by gradient descent and backpropagation. **By contrast, learning in the visual cortex is generally thought to occur primarily by local learning (with some influences from both downstream and upstream activity)**, presumably according to the **Hebbian principle of reinforcing connections that cause a cell to fire**. Investigating Hebbian learning in large hierarchical networks could therefore provide a more relevant model of the visual system, as well as offering new methods for unsupervised and self-supervised (label-free) learning.

There is a long history of research in hierarchical Hebbian networks. The Neocognitron (an early hierarchical model for translation- and scaling-tolerant object recognition) used an effectively **Hebbian learning rule** [Fukushima and Miyake, 1982]. The **HMAX** model used an “imprinting” learning method to approximate Hebbian learning in a large network in a computationally tractable manner [Serre et al., 2007]. The **VISNet model** also studied various forms of Hebbian learning with temporal dynamics to learn invariant representations [Wallis and Rolls, 1997, Rolls and Milward, 2000]. While these models and others provided rich insight, **their performance remained far from current deep learning techniques**. Some of this difference may result from the massive computational advances that supported (and were prompted by) the deep learning revolution, including both specialized hardware [Pinto et al., 2009, Krizhevsky et al., 2012] and software frameworks to interact **with this hardware** in an intuitive manner [Paszke et al., 2019, Abadi et al., 2015].

Another motivation for investigating Hebbian networks is that Hebbian learning can be generalized to implement many forms of learning, including sparse coding [Brito and Gerstner, 2016], K-means [Hu et al., 2014] and similarity-matching [Bahroun and Soltoggio, 2017]. However, pure Hebbian multi-layer networks suffer from poor performance. Notably, a consistent result is that decoding performance *decreases* over successive layers, suggesting loss of information [Amato et al., 2019, Bahroun et al., 2017]. This is a particularly disconcerting finding, since the purpose of hierarchical networks is precisely to aggregate information and produce more exploitable representations over successive layers [DiCarlo and Cox [2007], Serre et al. [2007].

Recently several authors have proposed using modern deep learning framework for Hebbian learning in multi-layer convolutional networks [Amato et al., 2019, Talloen et al., 2021]. These projects rely on intricate hand-crafted machinery to implement Hebbian learning. While Talloen et al. [2021] expose this machinery as a remarkable reusable framework, this necessarily constrains possible usage to whatever abilities the framework provides, in addition to burdening users with the need to learn the framework itself.

Here we show that Hebbian learning can be implemented very easily with modern deep learning frameworks, by using surrogate losses whose gradients turn out to produce exactly the desired Hebbian updates. As a result, we use the frameworks as they were intended to be used, with little additional burden, freeing us to concentrate on actual network design. Importantly, the actual outputs of the network are arbitrary and need not be differentiable, since the surrogate losses are only used to set up the computational graph. We provide expressions for losses whose gradients exactly implement several Hebbian rules, namely: plain Hebbian ($\Delta w \propto xy$), Grossberg’s instar rule ($\Delta w \propto y(x - w)$), and Oja’s rule ($\Delta w \propto y(x - yw)$).

As an application, we build simple convolutional multi-layer networks for object recognition, which we test on the CIFAR10 dataset. We observe that the higher layers of such networks do not spontaneously learn complex shapes. Rather, they simply learn large, simple features (Gabor-like oriented edge detectors and blobs), by combining lower-level features appropriately. This explains the reduction in decoding performance over successive layers, despite a higher number of filters, as was observed in previous work [Amato et al., 2019, Bahroun et al., 2017, Bahroun and Soltoggio, 2017]. To combat this tendency, we introduce several interventions (“triangle” method for computing activations [Coates et al., 2011] and massive pruning of connections between layers) which both prevent the formation of high-level Gabors and massively increase higher-level performance, allowing higher layers to produce more informative representations than the first layer. We hypothesize that incorporating more advanced features of previous models (dynamic stimuli with trace learning, feedback connections, etc.), together with the massive computational boost offered by modern deep learning frameworks, could greatly improve the performance and biological relevance of multi-layer Hebbian networks.

2 Methods

2.1 Hebbian learning with deep learning frameworks

Modern deep learning frameworks implement automatic differentiation with transparent access to GPU computation, batched processing, and convolutional filtering. As such, they are ideally suited to compute batch gradients over multi-layer convolutional networks [Paszke et al., 2019, Abadi et al., 2015].

Implementing Hebbian learning poses no difficulty with fully-connected layers, or fully recurrent networks, because the input and output are simple vectors (or matrices when considering the batch dimension). As a result, computing the Hebbian weight update (based on a product of inputs and outputs) requires little more than an outer vector product. No special machinery is needed [Miconi et al., 2018].

Things are not so simple with convolutional networks, because in this case the inputs to the weights are composed of *overlapping* patches within the previous layer’s output. As such, we can not simply grab the inputs and outputs and perform a straightforward outer product. We propose two methods, not necessarily exclusive, to implement such computations with deep learning frameworks.

2.1.1 Method 1: Hebbian updates as gradients of surrogate losses

By far the simplest method is to compute a *surrogate loss*, the gradient of which just happens to be the desired Hebbian weight update.

We first compute the “real” output of that layer, which is arbitrary and can involve any computations, differentiable or not (in this work we mostly use a winner-take-all competition, which is not straightforwardly differentiable). Then, we compute a *surrogate loss for this layer*, designed specifically to produce the desired update as a gradient (see Appendix A for equations and code samples). After this surrogate loss is computed, setting in place the appropriate computational graph, we overwrite the values of this surrogate output with the “real” output values previously computed, without

impacting the computational graph (because the gradient of the surrogate loss must be evaluated at the “real” value of the output). Then we simply invoke the framework’s backward pass function to compute and apply the desired weight updates (see Appendix A for details).

At first sight this seems to require two separate forward passes: one to compute the real output, and one to compute the surrogate loss. However, this is not the case, because the computationally intensive element (namely, the linear convolution by the weight filters) is common to both real and surrogate output, and thus only needs to be computed once. The only additional computation is the surrogate loss function, which is generally negligible in comparison to the convolution itself.

Equations and code samples for plain Hebb, Instar and Oja’s rule are provided in Appendix A. All code for the experiments described in this paper is available at <https://github.com/ThomasMiconi/HebbianCNNPyTorch>.

2.1.2 Method 2: Unfolding the input with fixed convolutions

Alternatively, we can eliminate overlapping input patches by separating the weight convolution into two successive convolutions: a fixed, binary convolution that rearranges every input patch into a single column vector, followed by a 1×1 convolution that contains the actual weights. More precisely, suppose our original convolution has input size $h \times w \times n_i$ (where h and w are the height and width of the convolutional filter, and n_i is the number of channels in the input), with n_o output channels. Then, we can first pass the input through a fixed convolution of input size $h \times w \times n_i$ with hwn_i output channels, with a fixed weight vector set to 1 for the weights that links input x, y, i to output xyi (where x, y and i run from 1 to h, w and n_i respectively) and 0 everywhere else. This rearranges (and duplicates) the values of each input patch of the original convolution into single, non-overlapping column vectors. Afterwards we can apply the actual weights of the original convolution with a simple 1×1 convolution, which can be performed by a simple tensor product with appropriate broadcasting if necessary.

This somewhat clunkier method does require two convolutional steps, as well as additional memory usage. However, it also provides finer-grained control. For example, it makes it easy to separately normalize and whiten each input patch of the convolution separately, which is commonly done in computer vision models [Coates et al., 2011, Olshausen and Field, 1997], but not easy to do with a normal convolution. In addition, it makes it possible to compute the Hebbian weight update by hand, since the inputs of each weight application are neatly separated into non-overlapping vectors. This may be of interest if one seeks more complex updates that are not easily reducible to a gradient. Conversely, note that this method and the previous one are not mutually incompatible: it is possible to use Method 2 to provide individual patch normalization (and whitening, see below), and then Method 1 to compute the actual weight updates.

All results reported in this work use Method 1 exclusively. In our experiments, individual patch normalization did not seem to improve performance (though individual patch whitening in the first layer did improve performance somewhat over the whole-image whitening method described below), and we are not studying any Hebbian rule beyond the ones mentioned above, making the added complexity of Method 2 superfluous.

2.2 Whitening

Natural images exhibit considerable correlations between pixels. It is generally desirable to remove this correlation in order to make the actual signal easier to extract [Olshausen and Field, 1997, Hyvärinen and Oja, 2000, Coates et al., 2011, Bahroun and Soltoggio, 2017]. In grayscale images, this correlation mostly takes the form of spatial autocorrelation, resulting in much higher energy at low frequencies; as a result, it is usually eliminated by a simple band-pass filter (in the spatial domain, such filters tend to resemble the center-surround structure of retinal receptive fields) [Olshausen and Field, 1997]. In color images, however, the correlations are more complex and involve inter-channel interactions. As a result, it is customary to use a more complex form of whitening, such as the so-called ZCA method which makes the covariance matrix of the transformed data equal to the identity matrix [Bell and Sejnowski, 1997, Krizhevsky and Hinton, 2009, Coates et al., 2011, Bahroun et al., 2017, Hyvärinen and Oja, 2000]. We first normalize each image vector $\mathbf{x}$ (i.e. subtract the mean and divide by the standard deviation¹), then multiply it by the matrix $\mathbf{E}\mathbf{D}^{-1/2}\mathbf{E}^T$, where $\mathbf{E}$ is the matrix of the eigenvectors of the covariance matrix of the $\mathbf{x}$ vectors and $\mathbf{D}$ the diagonal matrix of its eigenvalues (to which we add a regularization constant of 10^{-3} to avoid instabilities). Note that the spatial domain visualization of this filter also has a center-surround structure, though one which depends on the preferred color of the center and surround neurons (see e.g. Figure 1.3 in [Krizhevsky and Hinton, 2009]), suggesting a plausible biological implementation. In this work we first compute the ZCA matrix over the entire training set, then apply it to each input image during training and testing.

¹In theory, the mean and standard deviations are to be taken over the dataset. In practice, we found that taking them over each individual image works well, and indeed better than taking them over the batch dimension.

2.3 Network architecture

All experiments use the CIFAR10 dataset Krizhevsky and Hinton [2009], which contains 60000 color images of size 32×32 pixels (50000 for training and 10000 for testing). Each image is centered and whitened as explained above before being fed to the network.

Our network uses three convolutional layers, with kernel size 5×5 in the first layer and 3×3 in the others. Before each layer, we normalize each element in the batch by subtracting its mean and dividing it by its standard deviation (over all positions and channels).

Our basic network applies a winner-take-all selection (WTA) to each layer’s outputs: for each position, we only preserve the channel with the highest activation and set all other ones to 0 (the winning channel output is set to 1; while not strictly necessary, this binarizing makes learning more stable). This choice parallels previous work [Fukushima and Miyake, 1982, Masquelier et al., 2007] and is influenced by Hu et al. [2014]’s proof that Hebbian plasticity with WTA in the outputs is essentially equivalent to K-means clustering, which is known to produce highly efficient and biologically plausible receptive fields [Coates et al., 2011, Hu et al., 2014]. Later we describe different forms of selection that improve performance considerably over this baseline.

After each layer a 2×2 average pooling and downsampling is applied (note that plasticity occurs before pooling, so pooling at a layer has no effect on the learning in the same layer). We use relatively small networks with 100, 196 and 400 filters at layers 1, 2 and 3 respectively (196 is the closest perfect square to 200; we prefer square numbers of filters for presentation purposes). With these settings, receptive fields at layer 1, 2 and 3 have size 5×5 , 10×10 and 20×20 pixels, respectively. The final output of the network has dimensions $400 \times 2 \times 2$. We train the network over 20 epochs, each containing the whole training set; we verified that this duration was sufficient for stabilizing the learned weights.

In addition, we use adaptive thresholds to ensure similar firing rates across neurons: after computing the total input to each neuron, but before applying WTA, we add a bias term to each neuron’s activation that is adaptively modulated over time to ensure the firing rate of the neuron remains close to $1/N$, where N is the number of neurons in the layer (when using k -WTA, the target rate for each neuron is instead k/N). These thresholds are not strictly necessary, but help prevent the appearance of “dead” cells that never fire at all (though we note that such cells are in themselves an object of study [Shoham et al., 2006, Thorpe, 2011]).

We also keep the weight vectors of all neurons normalized to Euclidean norm 1. While Oja’s rule can theoretically constrain the norms of weight vector to one under some conditions, in practice we found that it did not fully stabilize weights in our settings (presumably due to interference from nonlinearities and adaptive thresholds).

In all experiments, we use Grossberg’s Instar rule [Vasilkoski et al., 2011], also used in previous work [Hu et al., 2014, Amato et al., 2019]. This is implemented by the surrogate gradient method described above. In practice, we found that with weights constrained to norm 1, the choice of plain Hebbian, Instar or Oja’s rule made little difference in the results.

2.4 Decoding and evaluation

We assess network performance with a simple linear decoder: we freeze weights, then feed the entire training set to the network and regress the one-hot label vectors over the network outputs. We then apply these regression weights to the network’s response to test set images, and pick the category with the largest output for each image. This simple regression classifier is orders of magnitude faster than the linear SVM used in other work [Coates et al., 2011, Bahroun et al., 2017], at only a minor performance cost.

To evaluate the information contained in each layer below the top one, we use the “quadrants” method introduced by Coates et al. [2011]. We divide the layer’s output spatially in four equal quadrants, average the value of each channel across all positions within each quadrant, and concatenate the results. For layer 1, this results in a vector of size 400 (100 filters times 4 quadrants), to which we apply the linear decoding procedure described above. We note that the quadrants method is an extremely powerful baseline, reaching a test accuracy of 43% on CIFAR10 from the output of the first layer of an *untrained* network, i.e. with randomly initialized weights (see Table 1). Note that for the last layer, because the network output has spatial dimensions 2×2 , the quadrants method is identical to simply using the output of the network as it is.

The receptive fields of layer 1, which take input directly from the RGB representation of input images, can be visualized by simply using the weights as image pixels (since weights can be negative, we normalize to the $[0, 1]$ range, ensuring that all receptive fields use a common value for weight 0). The receptive fields of higher levels are reconstructed as the weighted sums of previous-layer receptive fields at the appropriate location. This reconstruction is necessarily approximate, due to the pooling operation after each layer. Figures 1 and 2 show example receptive fields after training (full sets of learned receptive fields are shown in Figures B.1 and B.2).

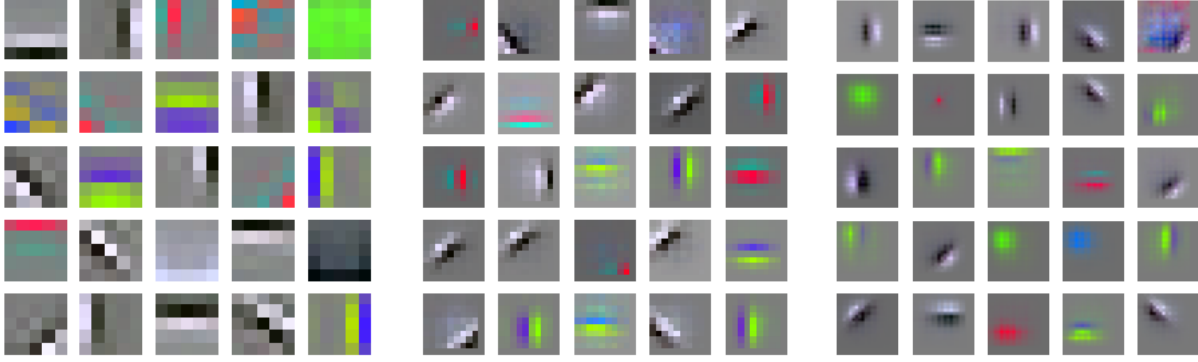


Figure 1: Example receptive fields from layers 1 (left), 2 (center) and 3 (right) in a standard Hebbian network with WTA competition. Receptive fields have size 5×5 (layer 1), 10×10 (layer 2) and 20×20 (layer 3). The full sets of receptive fields for each layer are shown in Figure B.1.

3 Results

3.1 Unconstrained hierarchical Hebbian learning builds simple higher-level features

We observe poor performance on CIFAR10, reaching a test accuracy of $34.5\% \pm 0.4$ (mean and standard deviation over 10 runs). As previously reported [Bahroun et al., 2017, Bahroun and Soltoggio, 2017, Amato et al., 2019], we observe that object information actually *decreases* across successive layers, despite the increasing number of channels. In particular, the quadrant methods applied to layers 1 and 2 result in test accuracies of $49.4\% \pm 0.4$ and $45.3\% \pm 0.6$, respectively (see Table 1 for a summary of results).

Inspection of learned receptive fields provides an explanation for this poor performance. As expected from previous work [Krizhevsky and Hinton, 2009, Bahroun et al., 2017], layer 1 receptive fields include a mixture of grayscale Gabor-like features, color-opponent cells (cyan-red, blue-orange and green-purple), and flat fields of white, black, red, green and blue (this outcome, including the specific colors, was extremely reproducible across many parameter and design choices). However, unlike surmised in traditional models of the visual system [Serre et al., 2007, Wallis and Rolls, 1997], the higher layers do not learn increasingly complex, shape-selective combinations of lower-level features. Instead, they simply learn larger Gabor-like features and poorly localized blobs. Notably, the layers do not simply perform an identity mapping, merely transmitting specific individual lower-level channels (as observed in the single-column case by Olshausen and Field [1997]); instead, they precisely combine lower-level features to form higher-level, larger oriented edge detectors and blobs. As a result, the information in higher layers is essentially a less spatially precise version of that provided by layer 1, making the loss of decoding performance unsurprising.

Varying the number of winners in each layer (that is, using k -WTA instead of WTA, for various values of k ranging from 1 to hundreds) had only minimal impact on performance, and simply resulted in larger, more blurry receptive fields.

3.2 Dense responses, pruned networks and dispersed weights improve performance

We introduce two manipulations which, put together, considerably improve performance and produce very different receptive fields.

1. We relax the competition between neurons for the purpose of computing activations, but *not* for the purpose of applying plasticity. Activations are now computed by the “triangle” method introduced by Coates et al. [2011]: at each position, we subtract the mean activation (across all channels at this position) from all channels, and rectify negative values to 0 (before pooling and downsampling). This results in less sparse, but still competitive responses: for each stimulus, on average, about half the cells at any given column have non-zero response. Importantly, this determines the responses transmitted to the next layer, but *not* the plasticity: the output used for plasticity at each position is still determined by binary WTA, as before (that is, only the most active neuron in any column undergoes plasticity). This differentiation between selection for activation and for plasticity considerably improved performance over alternative choices (e.g. WTA or k -WTA for both responses and plasticity, or triangle method for both).

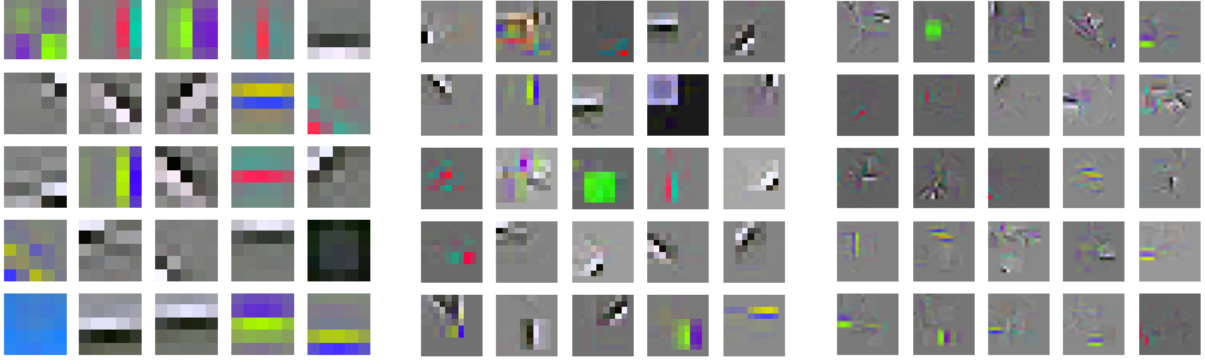


Figure 2: Example receptive fields from layers 1 (left), 2 (center) and 3 (right) in a Hebbian network with dense activations and pruned connections (plasticity still occurs through WTA competition). The full sets of receptive fields for each layer are shown in Figure B.2.

Table 1: Test accuracy on CIFAR10, for the default network (unpruned weights, WTA competition) and the pruned network with triangle method, before training (i.e. at initialization) and after Hebbian training on 20 epochs over the training dataset.

	L1 quadrants	L2 quadrants	Final output
Default network, untrained w	43.3% $\pm$ 0.7	21.05% $\pm$ 0.5	10.56% $\pm$ 0.4
Default network, trained w	49.94% $\pm$ 0.4	45.3% $\pm$ 0.6	34.5% $\pm$ 0.4
Network with triangle method & pruning, untrained w	52.38% $\pm$ 0.4	57.17% $\pm$ 0.3	56.3% $\pm$ 0.3
Network with triangle method & pruning, trained w	58.56% $\pm$ 0.3	64.2% $\pm$ 0.4	64.55% $\pm$ 0.4

2. We massively prune the connectivity of the network, such that only 1% of all possible connections can be nonzero. This is done by generating a random binary mask for each weight vector at the beginning of the experiment, containing 99% of zeros and 1% of ones, and multiplying the weight vectors by their mask after every weight modification (we still normalize the pruned weights to norm 1).

Note that these methods are only applied to layers 2 and above, since layer 1 is supposed to take inputs directly from the “retinal” image input.

These manipulations, put together, greatly improve performance. With these alterations, the model reaches a test accuracy of 64.55% $\pm$ 0.5 (mean and standard deviation over 10 runs). This is well above the decoding accuracy from the quadrants method on layer 1 (58.56% $\pm$ 0.3), and similar to the same from layer 2 (64.2 $\pm$ 0.3), showing that information now increases or persists over successive layers (see Table 1 for a summary of results).

4 Discussion

Transparent use of deep learning frameworks for multi-layer Hebbian learning opens the door to many avenues of exploration. In this work, we showed that unconstrained Hebbian learning tends to settle on simple, information-poor features, explaining the paradoxical reduction in information and decoding performance over successive layers that had been previously reported [Bahroun et al., 2017, Amato et al., 2019]. We countered this with a more permissive selection in activations (but not for plasticity), and a pruning of synaptic trees. However, these only scratch the surface of possibilities. Further possible directions of research include:

1. *Dynamic stimuli and temporal learning rules:* There is a long history of using the temporal structure of visual stimuli to extract transformation-invariant representations [Földiák, 1991, Wallis and Rolls, 1997, Masquelier et al., 2007]. In this work we have only considered static stimuli, but it would be highly desirable to exploit the wealth of temporal information available in natural stimuli.
2. *Decorrelation in higher layers:* Here we use ZCA decorrelation in the first layer, and rely on WTA to prevent highly correlated firing. More aggressive forms of decorrelation in the upper layers might prove beneficial. We experimented with channel-wise lateral inhibition and online ZCA, but neither improved performance. More elaborate (or more biologically inspired) scheme might produce different results.

3. *Feedback connections*: in the visual system, feedback connections carry information from upper to lower layers. The computational role of these connections has been debated [Lee and Mumford, 2003, Hupé et al., 1998, Rao and Ballard, 1999]. Recent computational work suggests that appropriately configured feedback can greatly improve learning [Payeur et al., 2021, Lindsey and Litwin-Kumar, 2020]. How to incorporate feedback connections into large networks remains an area of active research, which could greatly benefit from the computational power of deep learning frameworks.

We note that, although our interventions greatly improved the performance of the network, the learned features (see Figure 2) are difficult to interpret. They are also quite different from the organized, shape-selective features posited by hierarchical models of the visual cortex. In these models, lower layers are expected to respond to simple conjunctions of edges and angles, while intermediate layers respond to middle-scale components such as curvature, and the higher layers develop selectivity for individual objects and faces across multiple poses. This raises the question: how can biologically plausible hierarchical networks develop selectivities to specific, increasingly complex stimuli, elaborating their selectivities across successive layers in a way that matches the increasing specificity of visual cortex? We suggest that the computational power of modern deep learning frameworks is likely to prove extremely useful for such investigations.

In another direction, several meta-learning approaches seek to discover new learning rules for particular domains Bengio et al. [1997], Risi and Stanley [2010], Najarro and Risi [2020]. The method described above could conceivably be harnessed to search over possible loss expressions (as functions of x, y, w or any other variable accessible to the framework), seeking losses whose gradients would produce optimal learning, whether in unsupervised, self-supervised, or supervised learning.

In conclusion, we showed that modern deep learning frameworks are a powerful tool for exploring Hebbian learning in multi-layer convolutional networks. We believe these frameworks offer exciting opportunities for further research, both in terms of performance for unsupervised and self-supervised learning, and of biological relevance and insight for models of the visual cortex.

Appendix

A Hebbian updates with gradients

All code for the experiments described above is available at <https://github.com/ThomasMiconi/HebbianCNNPyTorch>.

We want to build a surrogate loss function, such that the gradient of this loss is equal to the desired Hebbian update. To do this, we note that the loss $\mathcal{L}$ is a function of the network output y , and thus its derivative over the weight w can be written as follows (by the chain rule):

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial y} \bigg|_{\hat{y}} \frac{\partial y}{\partial w}, \quad (1)$$

where $\hat{y}$ is the (“real”) numerical value of the output y . We can thus obtain our desired Hebbian updates as the gradients of the following expressions for $\mathcal{L}$ and y :

$$\mathcal{L} = \frac{1}{2} y^2 \quad (2)$$

$$y = \begin{cases} wx & \text{(Plain Hebb)} \\ wx - w^2/2 & \text{(Instar rule)} \\ wx - \hat{y}w^2/2 & \text{(Oja's rule)} \end{cases} \quad (3)$$

By using Equation 1, it is easily verified that the derivatives of the above expressions over w are $xy, y(x - w)$ and $y(x - \hat{y}w)$, respectively. We have also verified that the resulting gradients computed by the framework are equal to hand-computed Hebbian updates in the code.

The following code implements the method in PyTorch:

```
optimizer.zero_grad()
```

```

# Perform the weight convolution, setting up the first part
# of the computational graph.
prelimy = F.conv2d(x, w)

# We now compute the "real" output really, with a k-WTA
really = prelimy.clone().detach() # We don't want to affect the graph
tk = torch.topk(really.data, K, dim=1, largest=True)[0]
really.data[really.data < tk.data[:, -1, :, :][:, None, :, :]] = 0
really.data = (really.data > 0).float() # Binarizing

# We now compute the surrogate output y, used only to set up the
# proper computational graph.
yforgrad = prelimy # Plain Hebb, dw ~ = xy

# The following expressions implement the Instar rule (dw ~ = y(x-w)) and Oja's
# rule (dw ~ = y(x-wy)), respectively. Note the dimensional rearrangements and
# broadcasting.
# yforgrad = prelimy - 1/2 * torch.sum(w * w, dim=(1,2,3))[None, :, None, None]
# yforgrad = prelimy - 1/2 * torch.sum(w * w, dim=(1,2,3))[None, :, None, None] * really.data

# We overwrite the values of yforgrad with the "real" y.
yforgrad.data = really.data

# Compute the loss and perform the backward pass, which applies the
# desired Hebbian updates.
loss = torch.sum(-1/2 * yforgrad * yforgrad)
loss.backward()
optimizer.step()

```

B Full sets of learned receptive fields

Figure B.1 shows the full set of learned receptive field in the original, unconstrained Hebbian network with WTA competition. Figure B.2 shows the full set of learned receptive field in a Hebbian network with “triangle” activations and pruned connectivity (with plasticity still determined by WTA competition).

References

- Daniel LK Yamins and James J DiCarlo. Using goal-driven deep learning models to understand sensory cortex. *Nature neuroscience*, 19(3):356–365, 2016.
- Kuniyiko Fukushima and Sei Miyake. Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition. In *Competition and cooperation in neural nets*, pages 267–285. Springer, 1982.
- Thomas Serre, Aude Oliva, and Tomaso Poggio. A feedforward architecture accounts for rapid categorization. *Proceedings of the national academy of sciences*, 104(15):6424–6429, 2007.
- Guy Wallis and Edmund T Rolls. Invariant face and object recognition in the visual system. *Progress in neurobiology*, 51(2), 1997.
- Edmund T Rolls and T Milward. A model of invariant object recognition in the visual system: learning rules, activation functions, lateral inhibition, and information-based performance measures. *Neural computation*, 12(11):2547–2572, 2000.
- Nicolas Pinto, David Doukhan, James J DiCarlo, and David D Cox. A high-throughput screening approach to discovering good forms of biologically inspired visual representation. *PLoS Comput Biol*, 5(11):e1000579, 2009.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin

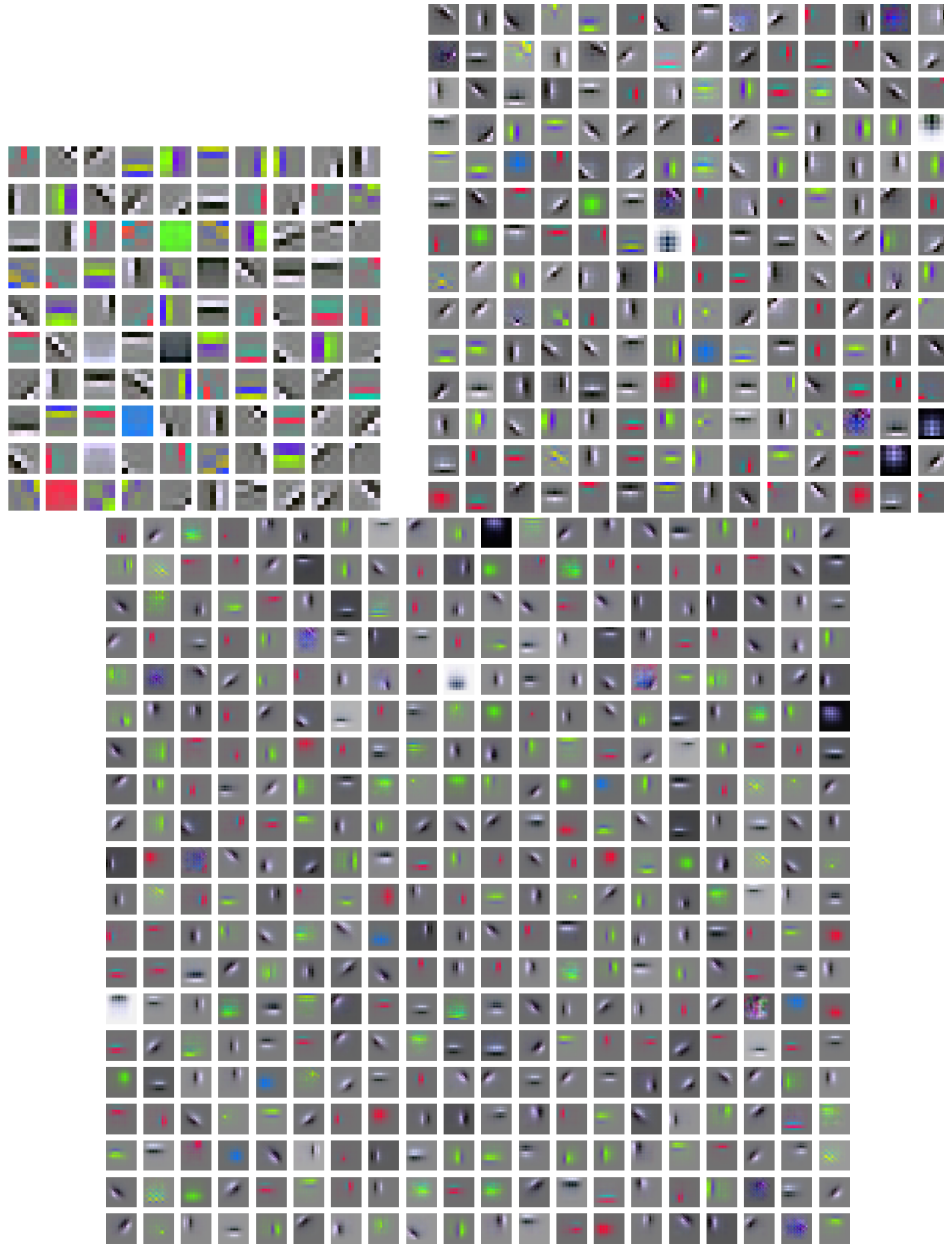


Figure B.1: Full set of learned receptive fields from layers 1, 2 and 3 in a standard Hebbian network with WTA competition.

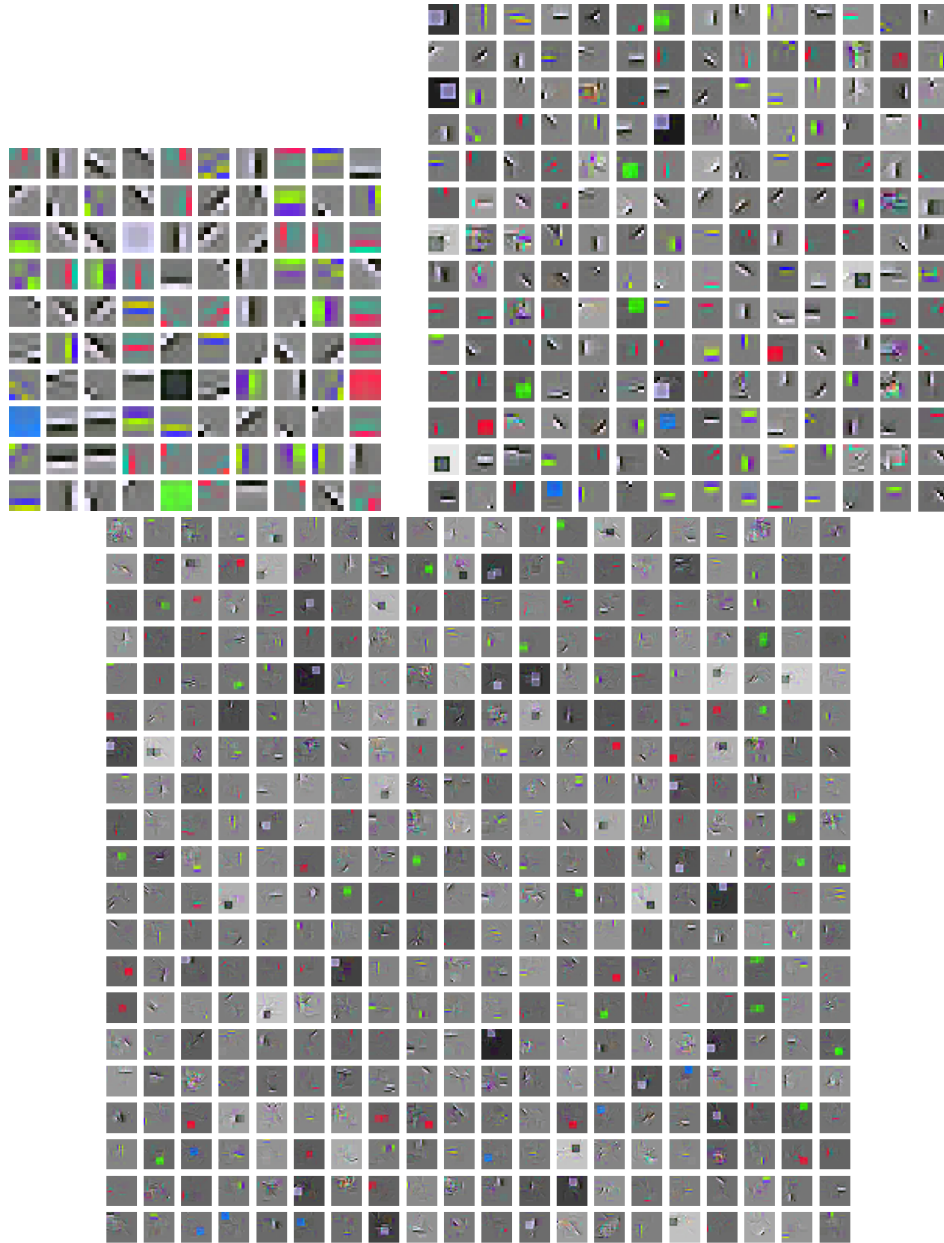


Figure B.2: Full set of learned receptive fields from layers 1, 2 and 3 in a Hebbian network with dense activations and pruned connections (plasticity still occurs through WTA competition).

- Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32, 2019.
- Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from tensorflow.org.
- Carlos SN Brito and Wulfram Gerstner. Nonlinear hebbian learning as a unifying principle in receptive field formation. *PLoS computational biology*, 12(9):e1005070, 2016.
- Xiaolin Hu, Jianwei Zhang, Peng Qi, and Bo Zhang. Modeling response properties of v2 neurons using a hierarchical k-means model. *Neurocomputing*, 134:198–205, 2014.
- Yanis Bahroun and Andrea Soltoggio. Online representation learning with single and multi-layer hebbian networks for image classification. In *International Conference on Artificial Neural Networks*, pages 354–363. Springer, 2017.
- Giuseppe Amato, Fabio Carrara, Fabrizio Falchi, Claudio Gennaro, and Gabriele Lagani. Hebbian learning meets deep convolutional neural networks. In *Image Analysis and Processing – ICIAP 2019*, pages 324–334. Springer International Publishing, 2019.
- Yanis Bahroun, Eugénie Hunsicker, and Andrea Soltoggio. Building efficient deep hebbian networks for image classification tasks. In *International Conference on Artificial Neural Networks*, pages 364–372. Springer, 2017.
- James J DiCarlo and David D Cox. Untangling invariant object recognition. *Trends in cognitive sciences*, 11(8):333–341, 2007.
- Jules Talloen, Joni Dambre, and Alexander Vandesompele. Pytorch-hebbian: facilitating local learning in a deep learning framework. *arXiv preprint arXiv:2102.00428*, 2021.
- Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In Geoffrey Gordon, David Dunson, and Miroslav Dudík, editors, *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, pages 215–223, 2011.
- Thomas Miconi, Kenneth Stanley, and Jeff Clune. Differentiable plasticity: training plastic neural networks with backpropagation. In *International Conference on Machine Learning*, pages 3559–3568, 2018.
- Bruno A Olshausen and David J Field. Sparse coding with an overcomplete basis set: A strategy employed by v1? *Vision research*, 37(23):3311–3325, 1997.
- Aapo Hyvärinen and Erkki Oja. Independent component analysis: algorithms and applications. *Neural networks*, 13(4-5):411–430, 2000.
- Anthony J Bell and Terrence J Sejnowski. The “independent components” of natural scenes are edge filters. *Vision research*, 37(23):3327–3338, 1997.
- A. Krizhevsky and G. Hinton. Learning multiple layers of features from tiny images. Master’s thesis, University of Toronto, Department of Computer Science, 2009.
- Timothee Masquelier, Thomas Serre, Simon Thorpe, and Tomaso Poggio. Learning complex cell invariance from natural videos: A plausibility proof. Technical Report MIT-CSAIL-TR-2007-060, Massachusetts Institute of Technology, 2007.
- Shy Shoham, Daniel H O’Connor, and Ronen Segev. How silent is the brain: is there a “dark matter” problem in neuroscience? *Journal of Comparative Physiology A*, 192(8), 2006.
- Simon J Thorpe. Grandmother cells, neocortical dark matter and very long term visual memories. *Journal of Vision*, 11(11):1243–1243, 2011.
- Zlatko Vasilkoski, Heather Ames, Ben Chandler, Anatoli Gorchetchnikov, Jasmin Léveillé, Gennady Livitz, Ennio Mingolla, and Massimiliano Versace. Review of stability properties of neural plasticity rules for implementation on memristive neuromorphic hardware. In *The 2011 International Joint Conference on Neural Networks*, pages 2563–2569, 2011.
- Peter Földiák. Learning invariance from transformation sequences. *Neural computation*, 3(2):194–200, 1991.
- Tai Sing Lee and David Mumford. Hierarchical bayesian inference in the visual cortex. *JOSA A*, 20(7):1434–1448, 2003.

- JM Hupé, AC James, BR Payne, SG Lomber, P Girard, and J Bullier. Cortical feedback improves discrimination between figure and background by v1, v2 and v3 neurons. *Nature*, 394(6695):784–787, 1998.
- Rajesh PN Rao and Dana H Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature neuroscience*, 2(1):79–87, 1999.
- Alexandre Payeur, Jordan Guerguiev, Friedemann Zenke, Blake A Richards, and Richard Naud. Burst-dependent synaptic plasticity can coordinate learning in hierarchical circuits. *Nat. Neurosci.*, 2021.
- Jack Lindsey and Ashok Litwin-Kumar. Learning to learn with feedback and local plasticity. *Adv. Neural Inf. Process. Syst. (NIPS 2020)*, 33, 2020.
- Samy Bengio, Yoshua Bengio, Jocelyn Cloutier, and Jan Gessei. On the optimization of a synaptic learning rule. In *Optimality in Biological and Artificial Networks?*, pages 281–303. Routledge, 1997.
- Sebastian Risi and Kenneth O Stanley. Indirectly encoding neural plasticity as a pattern of local rules. In *International Conference on Simulation of Adaptive Behavior*, pages 533–543. Springer, 2010.
- Elias Najarro and Sebastian Risi. Meta-learning through hebbian plasticity in random networks. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 20719–20731. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/file/ee23e7ad9b473ad072d57aaa9b2a5222-Paper.pdf>.